emphasis on the factuality and truthfulness of knowledge. In contrast, **MMLU** tests specialized knowledge from 57 domains, ranging from the humanities to the social sciences to STEM (science, technology, engineering and math).

5.2.2 Fact completion

Here we aim to evaluate LMs' knowledge in a setting that factors out knowledge-agnostic capabilities like language understanding/reasoning, and we use simple prompts to test single facts. Specifically, we construct a new dataset of such prompts based on facts from Wikidata, and evaluate LMs on it.

Problem setting. In fact completion, given a prompt that asks for a fact (e.g., "The capital of France is ___"), the task is to predict the completion of the prompt ("Paris"). This task is a natural language equivalent of the classical triple completion task studied for relational data (Bordes et al., 2013). Given an incomplete entity triple consisting of (subject, predicate, ?), the goal is to predict the missing object entity. Our evaluation metric is 5-shot Accuracy@K (K=1, 5), where Accuracy@K indicates whether one of the system's top K predictions matches the ground-truth label.

Datasets. Our fact completion setup described above was inspired by the LAMA probe dataset (Petroni et al., 2019). LAMA was an early work that established the method to probe LMs' knowledge through a fact completon prompt like "The capital of France is ___". However, the original LAMA dataset only covered ∼40 types of generic relational knowledge (e.g., place of birth). Here we curate a more diverse dataset to evaluate LMs.

Specifically, we identified 12 domains spanning the humanities (e.g., law), social sciences (e.g., economics), STEM (e.g., biomedicine), and other general facts. For each domain, we manually identified 2-7 Wikidata relations and engineered a prompt template which converted the triple into a natural-language-completion task. We picked relations which captured factual information highly relevant to and representative of the domain. This results in 86 relation types in total. The full list of relations and their corresponding prompts is available in Appendix E.2. We then downloaded all triples corresponding to these relations (using the January 2022 Wikidata dump). We removed triples where either the subject or object entity did not have a Wikipedia page. We sampled 1000 triples for each property to use as our benchmark. We observed that when cast as a natural language completion, a single triple may have multiple correct answers, as (1) a triple may be correctly completed by multiple different objects, or (2) a single object may be referenced by multiple different names. In evaluating models, we consider any generation corresponding to any alias of a permissible object entity to be correct.

5.3 Reasoning

To measure a model's reasoning capabilities, we evaluate it in both synthetic and realistic reasoning-centric scenarios. We specify a suite of synthetic tasks that probe core reasoning primitives (non-ampliative reasoning, ampliative reasoning, recursive hierarchy, state tracking; §5.3.1: REASONING-PRIMITIVES), largely isolating reasoning from language and knowledge. To build on these primitives, we then test models in realistic contexts (mathematical reasoning, code reasoning, legal reasoning, logical reasoning, structured data reasoning; §5.3.2: REALISTIC-REASONING) that bring together these primitives. Put together, these approaches clarify the extent to which models possess capacities crucial for reasoning and how this is of service in real use cases that depend on reasoning.

5.3.1 Reasoning primitives

While reasoning is usually assumed to involve transitions in thought (Harman, 2013), possibly in some non-linguistic format, we typically assess reasoning abilities (e.g., in adult humans) by means of explicit symbolic or linguistic tasks. Indeed, communication and argument may even be what reasoning is ultimately for (Mercier & Sperber, 2017). In order to distinguish reasoning from language and knowledge as much as possible, we focus here on relatively abstract capacities necessary for sophisticated text-based or symbolic reasoning divided into four categories: *non-ampliative* reasoning, *ampliative* reasoning, reasoning with *recur-*

sive hierarchy, and *state tracking*. The first division follows Peirce (1974): *non-ampliative* reasoning involves conclusions that are in some sense already present in the premises, whereas *ampliative* reasoning involve conclusions that are only guaranteed if we accept further assumptions that were not explicitly given.[50] The last two evaluate how generally the model can reason both recursively and over changing states.

These tasks involve only elementary language — in some cases a mix of natural language and mathematical symbols — and little in the way of factual world knowledge. Our assumption is that these abilities will be requisite for reasoning independent of what facts happen to hold, or what (natural or artificial) language the model happens to be using. We emphasize these evaluations are intended to be representative but not exhaustive.

Non-ampliative Reasoning We first test two basic primitives for non-ampliative reasoning: *pattern matching* to identify a relevant rule and *variable substitution* to apply the rule. To instantiate these primitives, we implement scenarios with abstract symbols following LIME (Wu et al., 2021). We further follow Clark et al. (2020) to test deduction that combines these primitives in natural language with simple words for variables and sentence templates for rules.

Ampliative Reasoning To measure ampliative reasoning, we use explicit rule induction and implicit function regression, which corresponds to making and applying claims about the likely causal structure for observations. For rule induction, we design and implement `rule_induct` inspired by the LIME induction tasks, where we provide two examples generated from the same rule string, and task the model with inferring the underlying rule. For function regression, we design and implement `numeracy_prediction`, which requires the model to perform symbolic regression given a few examples and apply the number relationship (e.g. linear) to a new input.

Recursive Hierarchy. To test the ability of language models to reason recursively over deep and long hierarchical dependencies, we use the generalized Dyck languages (*Dyck*, or $\mathcal{D}_n$). Dyck is the family of languages which embodies what it means to be a language with hierarchical structure (Suzgun et al., 2019b). Although such structures are believed to be essential to natural language, similar nested hierarchies also arise in other core reasoning domains (Dehaene, 2020) and involve a particularly subtle variety of pattern matching.

To instantiate a task with Dyck, we require the model to generate the sequence of the closing brackets of a $\mathcal{D}_n$ word without its last few closing brackets.[51] By virtue of the task formulation, every input example (i.e. $\mathcal{D}_n$ prefix) has a *unique* sequence of closing brackets. In our experiments, we limit our attention to a subset of the Dyck languages, $\mathcal{D}_3$, of well-nested strings over three pairs of brackets (viz., {"(", ")", "[", "]", "{", "}"}) and consider 500 evaluation examples of varying lengths (between 52 and 100) under a two-shot prompting protocol. To measure model accuracy, we report strict exact-match results.

State Tracking. To further evaluate the reasoning and state tracking capabilities of models, we measure their performance on **bAbI** (Weston et al., 2015). **bAbI** features short stories about characters picking and dropping items while traversing rooms in a house. Each question has a single-word answer, and they require a variety of reasoning skills, spanning transitive inference, coreference resolution, logical reasoning (negation, conjunctions and disjunctions), spatial and temporal reasoning, deduction and induction, grouped into 20 different tasks. The inputs have 64 tokens per instance on average, with significant variance in input lengths, with some reaching around hundreds of tokens.

5.3.2 Realistic reasoning

We also evaluate language models on more complex and realistic reasoning tasks that require multiple primitive reasoning skills. These evaluations bridge the gap between understanding reasoning in very controlled

[50] As a paradigmatic instance of non-ampliative reasoning, we investigate *deductive* patterns in which the conclusion could not be false as long as the premises are true. For ampliative reasoning we investigate (what are often called) *induction* and *abduction*, which characteristically require tacit (e.g., statistical or causal) assumptions to justify a conclusion.

[51] A model capable of recognizing Dyck needs to emulate a discrete pushdown automaton (PDA).

and synthetic conditions and the type of reasoning required in practical contexts. In particular, they also help surface how the abstract property of reasoning, while decomposed into similar primitives across domains, takes on different textures based on the domain (e.g. the reasoning required in legal contexts is quite different from the reasoning required in code contexts). Further, these realistic scenarios are central to reasoning: in-the-wild reasoning requires the ability to compose primitives in a variety of ways. To operationalize this measurement, we choose a diverse set of reasoning-intensive scenarios with clear ground truth, so that automated evaluation at scale is possible.

Mathematical Reasoning. We use **GSM8K** (Cobbe et al., 2020) and **MATH** (Hendrycks et al., 2021c) to test model performance in the context of math exams of varying difficulties. Both datasets evaluate multi-step mathematical reasoning capabilities necessary to arrive at a numerical answer given a question. In their ground truth examples, the datasets include intermediate steps in natural language which lead to the final answer, which the language model then imitates.

Code Synthesis. We use **HumanEval** (Chen et al., 2021) and **APPS** (Hendrycks et al., 2021a), which contain 164 and 10,000 coding questions respectively. **HumanEval** contains handwritten programming problems that tests simple algorithms and mathematics, while **APPS** is curated from open-access programming sites and covers a wider range of difficulties. Since each full coding task example could be too long to fit multiple of them in the context window, we only perform zero-shot evaluation with models fine-tuned on code already.

Legal Reasoning. For legal reasoning, we construct **LegalSupport**, a novel comparative legal entailment task. Lawyers often must determine which case most forcefully or accurately supports their claims. This is integral part of legal argument formulation, as lawyers must cite to the decisions made by previous courts (i.e. "precedent"). In particular, this allows one to (1) determine what the law says, and (2) persuasively argue for how a court should decide in a pending dispute. In **LegalSupport**, a model is provided with an argument, and the legal conclusions of two court cases. The task is to determine which case most persuasively supports the argument. The arguments and annotations are mined from actual legal opinions.

Logical Reasoning. We perform further evaluation on analytical reasoning questions from the Law School Admission Test (**LSAT**; Zhong et al., 2021), a standardized test for prospective law school candidates, which are verbal versions of constraint satisfaction problems (CSP) about assignments, groupings or orderings of elements in accordance with a list of requirements, like assigning several people to tables, or organizing classes in a schedule. The questions are of multi-answer format with 5 answers per question, where they either ask for the specific solution to the CSP, the cardinality of the solution space, or other derived properties and inferences about a given problem. The prompts have 170 tokens per example on average, and we provide 5 such examples in-context before asking the model to predict the answer to a new exercise.

Structured Data Reasoning. We evaluate how well models can wrangle structured data—a critical problem in enterprise data pipelines that has been studied in the data management community for over two decades (Golshan et al., 2017). We evaluate on two classic data integration and cleaning tasks: entity matching and data imputation. Entity matching is the task of determining if two structured rows (often from different relations) refer to the same entity. Data imputation is the task of filling in a missing cell value from a structured row. For entity matching, we use the Beer, Abt-Buy, and iTunes Amazon (dirty) datasets from the Magellan benchmark (Konda et al., 2016). For imputation, we choose the Restaurant and Buy datasets from Mei et al. (2021). For the tasks, we generate prompts following the random prompts from Narayan et al. (2022).

5.4 Memorization & copyright

Recent works have demonstrated that language models are capable of memorizing and reproducing content in training datasets (Carlini et al., 2019; Lee et al., 2022a; Carlini et al., 2022; Kandpal et al., 2022; Carlini et al., 2021). The ability to memorize and plagiarize suggests potential legal risks from an intellectual property standpoint (Bommasani et al., 2021, §5.4). In this section, we provide an evaluation of this risk